Método	**Criterio de determinismo**
`travel`	`sorted(self.adjacent_vertices(...))` na exploración de veciños
`connected_components`	`min(unvisited)` para seleccionar sempre o mesmo nodo de partida na busca de cada compoñente
`find_cut_vertices`	`sorted(components, key=lambda c: min(c))` para iterar compoñentes `min(component)` para a raíz da árbore
`path`	`sorted(self.adjacent_vertices(...))` na exploración de veciños desempate por tuplas `(distancia, nodo)` en `heapq` aproveitando a comparación por defecto de Python

Ademais, no **comando LOAD** (`src/graphs/commands/load.py`), cando o grafo recibido sexa `None`, debedes instanciar un novo obxecto Graph baleiro e engadir os nodos e as conexións usando os métodos `.add()` e `.connect()`.

Desta forma, os vértices insertaranse na mesma orde en que se len do ficheiro JSON, asegurando que a matriz imprimida polo comando PRINT manteña sempre a mesma orde de filas e columnas.